

```
!pip install pandas numpy scikit-learn rank_bm25 --quiet

import pandas as pd
import numpy as np
import re

from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.metrics.pairwise import cosine_similarity
from sklearn.preprocessing import MinMaxScaler

from rank_bm25 import BM25Okapi
```

```
from google.colab import files
uploaded = files.upload()
```

Choose Files

No file chosen

Upload widget is only available when the cell has been executed in the current browser session. Please rerun this cell to enable.

Saving dataset[1].csv to dataset[1].csv

```
import pandas as pd

df = pd.read_csv('dataset[1].csv')

print(df.shape)
df.head()
```

(114000, 21)

	Unnamed: 0	track_id	artists	album_name	track_name	popularity	duration_ms	explicit	danceability	energy
0	0	5SuOikwiRyPMVoIQDJUgSV	Gen Hoshino	Comedy	Comedy	73	230666	False	0.676	0.58
1	1	4qPNDBW1i3p13qLCt0Ki3A	Ben Woodward	Ghost (Acoustic)	Ghost - Acoustic	55	149610	False	0.420	0.35
2	2	1iJBSr7s7jYXzM8EGcbK5b	Ingrid Michaelson;ZAYN	To Begin Again	To Begin Again	57	210826	False	0.438	0.35
3	3	6lfxq3CG4xtTiEg7opyCyx	Kina Grannis	Crazy Rich Asians (Original Motion Picture Soundtrack)	Can't Help Falling In Love	71	201933	False	0.266	0.35
4	4	5vjLSffimilP26QG5WcN2K	Chord Overstreet	Hold On	Hold On	82	198853	False	0.618	0.58

5 rows × 21 columns

```
# -----
# 2. CLEAN DATA
# -----
df.dropna(subset=['track_name', 'artists'], inplace=True)
df.drop_duplicates(subset='track_id', inplace=True)
df.reset_index(drop=True, inplace=True)
```

```
# -----
# 3. CREATE COMBINED TEXT (IMPORTANT)
# Give more weight to genre
# -----
df['combined_text'] = (df['track_name'].fillna('') + ' ' +
                      df['artists'].fillna('') + ' ' +
                      df['track_genre'].fillna(''))
```

```
print('Cleaned dataset shape:', df.shape)
print('\nSample combined text:')
print(df['combined_text'].head(3))
```

Cleaned dataset shape: (89740, 12)

```
Sample combined text:
0          Comedy Gen Hoshino acoustic
1          Ghost - Acoustic Ben Woodward acoustic
2  To Begin Again Ingrid Michaelson;ZAYN acoustic
Name: combined_text, dtype: object
```

```
from sklearn.feature_extraction.text import TfidfVectorizer
# Step 5: Build TF-IDF matrix
# Use 20,000 songs for fast execution in Colab
sample_df = df.sample(20000, random_state=42).reset_index(drop=True)

# TfidfVectorizer converts text to TF-IDF feature matrix
# max_features=10000 keeps the 10,000 most common terms
# ngram_range=(1,2) captures single words AND two-word phrases
vectorizer = TfidfVectorizer(
    stop_words='english',    # remove common words like 'the', 'and'
    max_features=10000,
    ngram_range=(1, 2)      # use unigrams and bigrams
)

tfidf_matrix = vectorizer.fit_transform(sample_df['combined_text'])
print('TF-IDF matrix created successfully!')
print(f'Matrix shape: {tfidf_matrix.shape}') # (20000 songs, 10000 terms)
```

```
TF-IDF matrix created successfully!
Matrix shape: (20000, 10000)
```

```
from sklearn.metrics.pairwise import cosine_similarity
# Step 6: Define the TF-IDF search function
def tfidf_search(query, top_n=5):
    # Convert query to a TF-IDF vector
    query_vec = vectorizer.transform([query])

    # Compute cosine similarity between query and all songs
    similarity = cosine_similarity(query_vec, tfidf_matrix).flatten()

    # Add similarity scores to the dataframe
    results = sample_df.copy()
    results['score'] = similarity

    # Sort by score (highest first) and return top N
    results = results.sort_values(by='score', ascending=False)
    return results[['track_name', 'artists', 'track_genre', 'score']].head(top_n)

# Step 7: Test the search function
query = input('\nEnter song to search: ')
results = tfidf_search(query)
print('\nTop Results (TF-IDF):')
print(results.to_string(index=False))
```

Enter song to search: love

```
Top Results (TF-IDF):
  track_name  artists track_genre  score
Sirens of Love Paradoks      techno 0.719959
Hidden Love  Mokhov          idm 0.665004
Why You Love Me  Nosa      afrobeat 0.661984
Love You  Rewildz      hardstyle 0.661055
Love      Regain      hardstyle 0.661055
```

```
# Step 1: Install BM25 library
!pip install rank_bm25 --quiet
from rank_bm25 import BM25Okapi
import re

# Step 2: Define text preprocessing function for BM25
def preprocess_for_bm25(text):
    # Convert to lowercase
    text = str(text).lower()
    # Remove special characters and punctuation
    text = re.sub(r'[^a-zA-Z\s]', '', text)
    # Split into individual word tokens
    tokens = text.split()
    # Remove very short tokens (single letters)
```

```

tokens = [t for t in tokens if len(t) > 1]
return tokens

# Apply preprocessing to all songs in the sample
tokenized_corpus = [preprocess_for_bm25(text)
                     for text in sample_df['combined_text']]

print('Text preprocessing for BM25 complete.')
print(f'Sample tokens: {tokenized_corpus[0]}')

```

Text preprocessing for BM25 complete.
Sample tokens: ['feliz', 'cumpleaños', 'ferxxo', 'feid', 'latin']

```

# Step 3: Initialise the BM25 model with the tokenised corpus
bm25 = BM25Okapi(tokenized_corpus)
print('BM25 model created successfully!\n')

```

BM25 model created successfully!

```

# Step 4: Define BM25 search function
def bm25_search(query, top_n=5):
    # Tokenise the query using the same preprocessing
    tokenized_query = preprocess_for_bm25(query)

    # Get BM25 relevance scores for all documents
    scores = bm25.get_scores(tokenized_query)

    # Add scores to the dataframe
    results = sample_df.copy()
    results['score'] = scores

    # Sort by score and filter out irrelevant results (score <= 0)
    results = results[results['score'] > 0]
    results = results.sort_values(by='score', ascending=False)
    return results[['track_name', 'artists', 'track_genre', 'score']].head(top_n)

# Step 5: Test BM25 search
query = input('Enter product to search: ')
results = bm25_search(query)
print('\nTop Results (BM25):')
print(results.to_string(index=False))

```

Enter product to search: dance pop

Top Results (BM25):

track_name	artists	track_genre	score
Dance of the Mad Bastards	Pop Will Eat Itself	industrial	7.386448
Yellow	Coldplay	pop	6.675603
Dance Again	Selena Gomez	dance	6.469037
Chicken Dance	Kids Dance Party	kids	6.134587
Numb	Khan Bhaini	pop	6.106231

```

from sklearn.preprocessing import MinMaxScaler
# Content-Based Recommender using Audio Feature Vectors

# Step 1: Define audio features to use
AUDIO_FEATURES = ['danceability', 'energy', 'acousticness',
                  'valence', 'tempo', 'instrumentalness', 'speechiness']

# Step 2: Normalise all features to 0-1 range
# This prevents tempo (0-200 BPM) from dominating over
# other features that are already on a 0-1 scale
scaler = MinMaxScaler()
sample_df[AUDIO_FEATURES] = scaler.fit_transform(sample_df[AUDIO_FEATURES])

# Step 3: Define the recommendation function
def content_recommend(liked_songs, top_k=5):
    liked = sample_df[sample_df['track_name'].isin(liked_songs)]
    if liked.empty:
        print('Songs not found. Try different names.')
        return
    # Build user profile as average of liked songs' feature vectors
    user_profile = liked[AUDIO_FEATURES].mean().values.reshape(1, -1)

```

```

# Compute cosine similarity against all songs
scores = cosine_similarity(user_profile, sample_df[AUDIO_FEATURES].values).flatten()
results = sample_df.copy()
results['score'] = scores
# Remove songs user already liked
results = results[~results['track_name'].isin(liked_songs)]
results = results.sort_values(by='score', ascending=False)
return results[['track_name', 'artists', 'track_genre', 'score']].head(top_k)

# Step 4: Test the recommender
print(content_recommend(['Blinding Lights', 'Levitating'], top_k=5))

```

	track_name	artists	track_genre	\
13318	Haru ni yuraredo kimi omou	kobasolo;kopi	anime	
14738	Twist Me Up	Audio Bullys	breakbeat	
3370	De Vloer Is Lava	DJ Maurice;Snollebollekes	hardstyle	
12167	Tlatelolco 68	Banda Bostik	metal	
4946	Ok Jaanu Title Track	A.R. Rahman;Srinidhi Venkatesh	pop-film	

	score
13318	0.999715
14738	0.999676
3370	0.999662
12167	0.999523
4946	0.999521